

دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

پیاده سازی و تحلیل سیاست های جایگزینی حافظه نهان در ChampSim

تحلیل و بررسی سیاست های جایگزینی پایه و پیشرفته و مقایسه آنها با یکدیگر

گروه ۱۳

امیررضا دولتی ۴۰۳۱۰۵۲۲۲
سینا فعالیت ۴۰۳۱۰۶۴۲۱

امیر محمد محقق ۴۰۳۱۰۶۶۴۶
سپهر کلانکی ۴۰۳۱۰۶۵۴۹

استاد درس: نام استاد
درس: معماری کامپیوتر

تابستان ۱۴۰۴

فهرست مطالب

۱	مقدمه	۱
۲	توضیح Replacement Policy ها	۲
۲-۱	یک نکته ی مشترک بین چند policy: نوع دسترسی access_type::WRITE	۲
۲-۲	توابع مهم برای هر سیاست	۲
۲-۳	LRU	۲
۲-۳-۱	قطعه ۱ - ساختار داده ی کمکی و سازنده	۲
۲-۳-۲	قطعه ۲ - انتخاب بلوک قربانی	۳
۲-۳-۳	قطعه ۳ - درج و به روزرسانی	۳
۲-۴	FIFO	۳
۲-۴-۱	قطعه ۱ - ساختار داده	۴
۲-۴-۲	قطعه ۲ - انتخاب قربانی و ثبت زمان ورود	۴
۲-۴-۳	قطعه ۳ - تابع خالی به روزرسانی	۴
۲-۵	MRU	۵
۲-۵-۱	قطعه ۱ - تنها تفاوت با LRU	۵
۲-۶	Random	۵
۲-۶-۱	قطعه ۱ - مولد شبه تصادفی و بازه ی انتخاب	۵
۲-۶-۲	قطعه ۲ - انتخاب قربانی و نبود توابع دیگر	۵
۲-۷	SRRIP	۶
۲-۷-۱	قطعه ۱ - helper هر set و مقداره ی اولیه	۶
۲-۷-۲	قطعه ۲ - انتخاب قربانی و افزایش گروهی	۷
۲-۷-۳	قطعه ۳ - به روزرسانی مشترک برای hit و fill	۷
۲-۸	BIP	۷
۲-۸-۱	قطعه ۱ - مولد تصادفی و پارامتر ϵ	۸
۲-۸-۲	قطعه ۲ - انتخاب قربانی (کاملاً مثل LRU)	۸
۲-۸-۳	قطعه ۳ - درج دو حالت	۹
۲-۹	DRRIP	۹
۲-۹-۱	قطعه ۱ - ثابت ها و شمارنده ی PSEL	۱۰
۲-۹-۲	قطعه ۲ - دو sub-policy درج	۱۰
۲-۹-۳	قطعه ۳ - انتخاب sub-policy در زمان درج	۱۱
۲-۹-۴	قطعه ۴ - hit و انتخاب قربانی	۱۱
۲-۱۰	SHiP	۱۲
۲-۱۰-۱	قطعه ۱ - ساختار sampler و جدول SHCT	۱۳
۲-۱۰-۲	قطعه ۲ - آموزش جدول از روی sampler	۱۳
۲-۱۰-۳	قطعه ۳ - استفاده از پیش بینی در زمان درج	۱۵
۲-۱۰-۴	قطعه ۴ - انتخاب قربانی	۱۵
۳	توضیح Trace ها	۱۶
۴	نحوه ی اجرا و خروجی گرفتن پروژه	۱۷
۱۸	مراجع	۱۸

چکیده

پر شود

12.5M warmup 50M simulation

توضیح Replacement Policy ها

برای تمام Replacement Policy ها یک دور روش آنها را به طور کامل توضیح می‌دهیم که گزارش کامل باشد. نکات ناقص رو نیز باید اضافه بکنیم.

۲-۱ یک نکته ی مشترک بین چند policy: نوع دسترسی `access_type::WRITE`
در چند تا از سیاست های، یک شرط روی `access_type` می بینیم که دسترسی های نوع `WRITE` را جدا می کند. این نوع دسترسی ها مربوط به `writeback` می شود. نکته این است که این نوشتن نشانه ی استفاده ی پردازنده از داده در همان لحظه نیست، بلکه صرفا نتیجه ی `replacement` در سطح بالاتر است. اگر `writeback` ها را مثل دسترسی عادی حساب کنیم و سیاست ها را روی آنها نیز اجرا کنیم، دو مشکل پیش می آید: اول اینکه بلوک هایی که فقط نوشته شده اند بی دلیل تازه به حساب می آیند و در کث ماندگار می شوند و دوم اینکه در سیاست های پویا مثل `DRRIP` آمار تصمیم گیری منحرف می شود چون این دسترسی ها رفتار واقعی برنامه را نشان نمی دهند. به همین دلیل در سیاست های ما یا اثر `writeback` نادیده گرفته می شود یا بلوک با یک مقدار امن و غیر مخرب پر می شود.

۲-۲ توابع مهم برای هر سیاست

- تابع `find_victim`: وقتی `miss` رخ داد، شماره ی `way` قربانی را برمی گرداند.
- تابع `replacement_cache_fill`: بعد از اینکه بلوک جدید داخل `way` قرار گرفت صدا زده می شود (منطق `fill`).
- تابع `update_replacement_state`: در هر دسترسی صدا زده می شود و پارامتر `hit` مشخص می کند دسترسی `hit` بوده یا نه.

نکته ی نهایی این نیز که بیشتر تنظیمات کلی و `global` به ازای هر `CPU Core` است، مثلا برای شمارنده ها و ...

۲-۳ LRU

منطق انتخاب بلوک قربانی:

بین تمام بلوک های موجود در یک `set`، بلوکی حذف می شود که بیشترین زمان بدون استفاده مانده است، به همین دلیل بلوک های که دسترسی بیشتری در طول زمان دارند کمتر حذف می شوند.

نحوه بروزسانی اطلاعات کمکی:

هر بلوک یک مقدار زمان دسترسی نگه می دارد که نشان می دهد نسبت به بقیه بلوک ها داخل همان `set` کی مورد دسترسی واقع شده است. مقدار `h` یعنی اینکه نسبت به بقیه قدیمی تر استفاده شده است، و هرچه مقدار آن بزرگتر باشد یعنی اینکه نسبت به بقیه جدید تر مورد دسترسی واقع شده است، و جدیدترین دسترسی این `set` است. هر دسترسی با توجه به `Miss` یا `Hit` شدن مقادیر را تغییر می دهیم که ترتیب دسترسی حفظ شود.

نکات کد:

۲-۳-۱ – ساختار داده ی کمکی و سازنده

```
1 class lru : public champsim::modules::replacement
2 {
3     long NUM_WAY;
4     std::vector<uint64_t> last_used_cycles;
5     uint64_t cycle = 0;
6     ...
7 };
8
9 lru::lru(CACHE* cache) : lru(cache, cache->NUM_SET, cache->NUM_WAY) {}
10
11 lru::lru(CACHE* cache, long sets, long ways)
```

```
12 : replacement(cache), NUM_WAY(ways),
13 last_used_cycles(static_cast<std::size_t>(sets * ways), 0) {}
```

اطلاعات اضافه در یک بردار یک-بعدی به طول $sets * ways$ نگه داری می شود و آدرس هر بلوک با فرمول $set * NUM_WAY + way$ به دست می آید.

۲-۳-۲ قطعه ۲ – انتخاب بلوک قربانی

```
1 long lru::find_victim(uint32_t triggering_cpu, uint64_t instr_id, long
  set, ...)
2 {
3     auto begin = std::next(std::begin(last_used_cycles), set * NUM_WAY);
4     auto end = std::next(begin, NUM_WAY);
5
6     // Find the way whose last use cycle is most distant
7     auto victim = std::min_element(begin, end);
8     assert(begin <= victim);
9     assert(victim < end);
10    return std::distance(begin, victim);
11 }
```

ابتدا با `begin` و `end` بازه ی مربوط به همان `set` جدا می شود و بعد `min_element` کوچک ترین برچسب زمان را پیدا می کند. چون برچسب بزرگ تر یعنی دسترسی جدیدتر. کوچک ترین مقدار دقیقاً یعنی بلوکی که از همه دیرتر استفاده شده است.

اگر چند بلوک مقدار برابر داشته باشند (مثلاً چند `way` هنوز دست نخورده و برابر ه باشند)، `min_element` اولین آنها را برمی گرداند، پس ترتیب پر شدن `set` از `way` کوچک به بزرگ است.

۲-۳-۳ قطعه ۳ – درج و به روزرسانی

```
1 void lru::replacement_cache_fill(uint32_t triggering_cpu, long set, long
  way, ...)
2 {
3     // Mark the way as being used on the current cycle
4     last_used_cycles.at((std::size_t)(set * NUM_WAY + way)) = cycle++;
5 }
6
7 void lru::update_replacement_state(..., uint8_t hit)
8 {
9     // Mark the way as being used on the current cycle
10    if (hit && access_type{type} != access_type::WRITE) // Skip this for writeback
    hits
11    last_used_cycles.at((std::size_t)(set * NUM_WAY + way)) = cycle++;
12 }
```

در زمان درج بلوک جدید در موقعیت MRU قرار می گیرد و کم خطر ترین بلوک `set` است.

FIFO ۲-۴

منطق انتخاب بلوک قربانی:

بین تمام بلوک‌های موجود در set، بلوکی که زودتر از همه وارد شده است باید خارج شود (منطق First In First Out). اینکه بلوک اخیراً دسترسی قرار گرفته یا نه تاثیری در تصمیم‌گیری ندارد. عملاً منطق صف دارد. نحوه بروزرسانی اطلاعات کمکی:

یک ترتیب ورود برای بلوک‌هایی که به set وارد می‌شوند ذخیره می‌کند. موقع hit چیزی را تغییر نمی‌دهد ولی موقع miss ترتیب عوض می‌شود (البته بدون آپدیت کردن مقدار بقیه چون زمان اضافه شدن اهمیت دارد). نکات کد:

۲-۴-۱ - ساختار داده

```
1 class fifo : public champsim::modules::replacement
2 {
3     long NUM_WAY;
4     std::vector<uint64_t> insertion_cycles;
5     uint64_t cycle = 0;
6     ...
7 };
```

ساختار داده دقیقاً مثل LRU است و تنها چیزی که فرق می‌کند مفهوم عدد کمکی است، که اینجا مفهوم زمان وارد شدن بلوک به set را می‌دهد. cycle در اینجا یک شمارنده کلی است که برای زمان اضافه شدن استفاده می‌شود و بیانگر شمارشگر زمان است که در هنگام هر fill افزایش پیدا می‌کند.

۲-۴-۲ - انتخاب قربانی و ثبت زمان ورود

```
1 long fifo::find_victim(uint32_t triggering_cpu, uint64_t instr_id, long
   set, ...)
2 {
3     auto begin = std::next(std::begin(insertion_cycles), set * NUM_WAY);
4     auto end = std::next(begin, NUM_WAY);
5
6     // Evict the oldest inserted block
7     auto victim = std::min_element(begin, end);
8     return std::distance(begin, victim);
9 }
10
11 void fifo::replacement_cache_fill(uint32_t triggering_cpu, long set, long way, ...)
12 {
13     // Record insertion time
14     insertion_cycles.at((std::size_t)(set * NUM_WAY + way)) = cycle++;
15 }
```

بدون اینکه صف واقعی داشته باشیم، با یک برچسب زمان ورود همان رفتار صف را می‌سازیم، کمترین مقدار یعنی قدیمی‌ترین ورود که همان بلوک از صف خارج می‌شود.

۲-۴-۳ - تابع خالی به روزرسانی

```
1 void fifo::update_replacement_state(..., uint8_t hit)
2 {
3 }
```

تابع update این policy خالی است چون طبق توضیحات در FIFO لازم نیست کاری انجام بدهیم.

۲-۵ MRU

منطق انتخاب بلوک قربانی:

کاملاً مخالف LRU عمل می کند ولی منطق مشابهی دارد به این صورت که بلوکی در set خارج می شود که نسبت به بقیه اخیراً دسترسی داشته است.

نحوه بروزرسانی اطلاعات کمکی:

مشابه LRU و نکته ای اضافه ای ندارد.

نکات کد:

۲-۵-۱ - تنها تفاوت با LRU

```
1 long mru::find_victim(uint32_t triggering_cpu, uint64_t instr_id, long
  set, ...)
2 {
3     auto begin = std::next(std::begin(last_used_cycles), set * NUM_WAY);
4     auto end = std::next(begin, NUM_WAY);
5
6     auto victim = std::max_element(begin, end); //MRU!!!
7     return std::distance(begin, victim);
8 }
```

کل تفاوت پایاده سازی با LRU استفاده از max_element به جای min_element است، به همین دلیل بلوکی که اخیراً به set اضافه شده است قربانی است. بقیه ی کد ها با LRU کاملاً یکسان هستند.

۲-۶ Random

منطق انتخاب بلوک قربانی:

در هر set به صورت رندوم یک بلوک را انتخاب می کنیم.

نحوه بروزرسانی اطلاعات کمکی:

اطلاعات اضافه ای برای ذخیره لازم نداریم.

نکات کد:

۲-۶-۱ - مولد شبه تصادفی و بازه ی انتخاب

```
1 struct random : public champsim::modules::replacement {
2     std::mt19937_64 rng{};
3     std::uniform_int_distribution<long> dist;
4     ...
5 };
6
7 random::random(CACHE* cache) : random(cache, cache->NUM_WAY) {}
8
9 random::random(CACHE* cache, long ways) : replacement(cache), dist(0, ways - 1) {}
```

رندوم روی بازه ی [0, ways-1] یک عدد انتخاب می کنیم که بیانگر آن بلوکی است که باید حذف شود.

۲-۶-۲ - انتخاب قربانی و نبود توابع دیگر

```
1 long random::find_victim(uint32_t triggering_cpu, uint64_t instr_id, long
  set, ...)
```



```

2  {
3      return dist(rng);
4  }

```

در سیاست random کل خطی که منطق این سیاست را پایاده سازی می کند همین است و توابع دیگر نیاز به override کردن ندارند چون این سیاست منطق هندل کردن حالت ندارد.

SRRIP ۲-U

به ازای هر بلوک در set یک عدد به نام RRPV (Re-Reference Prediction Value) ذخیره می شود که معمولاً ۲ بیت است که ۴ مقدار می تواند به خود بگیرد. در صورتی که ۰ باشد یعنی اینکه احتمالاً به زودی استفاده می شود و در صورتی که ۳ باشد احتمالاً خیلی در آینده ی دور استفاده می شود و در آینده ی نزدیک احتمالاً استفاده نمی شود.

منطق انتخاب بلوک قربانی:

در روش SRRIP (Static Re-Reference Interval Prediction) کل منطق بر این است که در یک set بلوک هایی که خیلی استفاده می شوند پایدار می شوند و در صورتی که استفاده ی یک بلوک کم شود احتمالاً بلوک قربانی خواهد بود. برای پیدا کردن بلوک قربانی هم بلوکی که RRPV برابر ۳ (بیشترین مقدار) را دارد خارج می شود. و در صورتی که هیچ بلوکی مقدار ۳ نداشت، مقدار RRPV تمام بلوک ها را زیاد می کنیم تا یکی به ۳ برسد، و این فرایند تا شرط برقرار شود ادامه پیدا می کند. (بخشی از بروزرسانی هم انجام می دهد)

نحوه بروزرسانی اطلاعات کمکی:

در صورتی که بلوک جدید اضافه شود، یک بلوک با RRPV تقریباً میانه مثل MAX_RRPV-1 اضافه می کند. در هنگام hit مقدار RRPV آن بلوک به ۰ ریست می شود. مقدار بروزرسانی هنگام خارج شدن یک بلوک هم داریم که بالا توضیح دادیم.

نکات کد:

۲-U-۱ قطعه ۱ – helper هر set و مقداردهی اولیه

```

1  struct srrip_set_helper {
2      using rrpv_type = int;
3      static constexpr rrpv_type maxRRPV = 3;
4
5      std::vector<rrpv_type> rrpv_values;
6      rrpv_type& get_rrpv(long way);
7
8      long victim();
9      void update(long way, bool hit);
10 };
11
12 srrip_set_helper::srrip_set_helper(long ways)
13     : rrpv_values(static_cast<std::size_t>(ways), maxRRPV) {}
14
15 srrip::srrip(CACHE* cache, long sets_, long ways_) : replacement(cache)
16 {
17     std::generate_n(std::back_inserter(sets), sets_,
18                     [ways = ways_] { return srrip_set_helper{ways}; });
19 }

```

برای داده های کمکی یک شی `srrip_set_helper` تعریف می کنیم که داده های مورد نیاز را به صورت گروهی نگه دارد.

مقدار اولیه ی همه ی `way` ها `maxRRPV` است، یعنی کش خالی طوری رفتار می کند که انگار همه ی بلوک ها در مرز خروج هستند. با این حرکت `way` های خالی بدون هیچ بررسی اضافه ای اول از همه انتخاب می شوند. در اینجا ما `maxRRPV` را مساوی ۳ قرار دادیم که بیانگر همان ۲ بیت برای این داده است.

۲-۱-۲ - انتخاب قربانی و افزایش گروهی

```
1 long srrip_set_helper::victim()
2 {
3     // Find the maximum RRPV
4     auto victim = std::max_element(std::begin(rrpv_values), std::end(rrpv_values));
5
6     // If the maximum element has RRPV less than the maximum, increment everything to
    the maximum
7     std::transform(std::cbegin(rrpv_values), std::cend(rrpv_values),
8         std::begin(rrpv_values),
9         [diff = maxRRPV - *victim](auto x) { return x + diff; });
10
11     // Return the way index
12     return std::distance(std::begin(rrpv_values), victim);
13 }
```

همان توصیف حلقه شکلی که در توضیح سیاست داشتیم اینجا به صورت یک عبارت جبری ساده به صورت `diff = maxRRPV - max` است که به همه اضافه می شود. عملاً همان کار حلقه را می کند و این دو روش هم ارزند ولی این بار محاسباتی کمتری دارد.

اگر از قبل بلوکی با مقدار بیشینه وجود داشته باشد، `diff` برابر صفر می شود و `transform` عملاً هیچ تغییری نمی دهد. یعنی همان حالت عادی که مستقیماً قربانی پیدا می شود.

نکته ی دیگر این است که این افزایش فقط در مسیر `miss` اتفاق می افتد.

۲-۱-۳ - به روزرسانی مشترک برای `hit` و `fill`

```
1 void srrip_set_helper::update(long way, bool hit) { get_rrpv(way) = hit ?
    0 : (maxRRPV - 1); }
2
3 void srrip::update_replacement_state(..., uint8_t hit)
4 {
5     sets.at(static_cast<std::size_t>(set)).update(way, hit);
6 }
```

کد `replacement_cache_fill` بازنویسی نشده است و هر دو حالت `hit` و `fill` از همین یک تابع استفاده می کنند که برای اینکه بین `hit` و `fill` تشخیص دهیم یک `bool hit` داریم که بین این دو انتخاب کند.

در صورتی که `hit` بشود مقدار به ۰ ریست می شود و در صورتی که `fill` باشد مقدار جدید برابر `maxRRPV-1` خواهد بود.

BIP ۲-۱

منطق انتخاب بلوک قربانی:

روش `BIP` (Bimodal Insertion Policy) عملاً روی روش `LRU` سوار شده است که منطق خارج کردن بلوک از `set` آن دقیقاً مشابه `LRU` است. صرفاً نحوه ی هندل کردن ترتیب بلوک ها بر اساس دسترسی اخیر آنها کمی متفاوت است.

نحوه بروزرسانی اطلاعات کمکی:

در روش LRU در هنگامی که بلوک جدید اضافه می‌شود، نسبت به بقیه در ترتیب دسترسی بالاتر بود. ولی در این روش هنگامی که بلوک جدید اضافه می‌شود با یک احتمال مشخص زیادی در ترتیب دسترسی پایینترین خواهد بود (Least Recently Used) که یعنی در خطر خارج شدن است (بر خلاف روش عادی که بلوکی که جدید اضافه می‌شد کم خطر ترین برای خارج شدن بود) و با یک احتمال متمم مشخص دیگری در ترتیب دسترسی نسبت به بقیه بالاترین خواهد بود (مثل حالت عادی روش LRU).

نکات کد:

۱-۲-۲ قطعه ۱ – مولد تصادفی و پارامتر ε

```
1  class bip : public champsim::modules::replacement
2  {
3      long NUM_WAY;
4      std::vector<uint64_t> last_used_cycles;
5      uint64_t cycle = 0;
6      std::mt19937 rng;
7      std::uniform_int_distribution<int> dist;
8      ...
9  };
10
11 bip::bip(CACHE* cache, long sets, long ways)
12     : replacement(cache),
13       NUM_WAY(ways),
14       last_used_cycles((std::size_t)(sets * ways), 0),
15       rng(0),
16       dist(0, 31)          // 1/32 probability
17 {
18 }
```

طبق توزیع $\text{dist}(0, 31)$ و شرطی که در قطعه ی سوم می‌آید ($0 ==$) یعنی احتمال درج در موقعیت MRU دقیقاً $1/32$ و احتمال درج در موقعیت LRU برابر $31/32$ است. (این همان احتمال هایی است که در توضیح این policy شرح دادیم) این مقدار به صورت رسمی ε نام دارد و انتخاب آن دلخواه نیست و باید هوشمندانه باشد، ولی مقدار معمولی آن همین است. باید آنقدر کوچک باشد که برای جست و جوی خطی حافظه باعث cache pollution نشود (داده های الکی و اضافه پر نکند) و آنقدر بزرگ باشد که اگر working set برنامه عوض شد، cache بتواند محتوای جدید را کم بپذیرد.

برای seed برای رندوم هم $\text{rng}(0)$ صفر را می‌دهیم.

بقیه ی ساختار داده دقیقاً همان LRU است.

۲-۲-۲ قطعه ۲ – انتخاب قربانی (کاملاً مثل LRU)

```
1  long bip::find_victim(uint32_t, uint64_t, long set, ...)
2  {
3      auto begin = std::next(last_used_cycles.begin(), set * NUM_WAY);
4      auto end = std::next(begin, NUM_WAY);
5
6      auto victim = std::min_element(begin, end);
```

```
7     return std::distance(begin, victim);
8 }
```

کاملاً مشابه LRU برای قربانی عمل می‌کنیم و منطق پیشرفته تری ندارد.

۲-۱-۳ قطعه ۳ - درج دو حالت

```
1 void bip::replacement_cache_fill(uint32_t, long set, long way, ...)
2 {
3     auto idx = (std::size_t)(set * NUM_WAY + way);
4
5     // 1/32 insert at MRU
6     if (dist(rng) == 0)
7     {
8         last_used_cycles[idx] = cycle++;
9     }
10    else
11    {
12        // Insert as LRU
13        last_used_cycles[idx] = 0;
14    }
15 }
16
17 void bip::update_replacement_state(..., uint8_t hit)
18 {
19     if (hit && type != access_type::WRITE)
20     {
21         last_used_cycles[(std::size_t)(set * NUM_WAY + way)] = cycle++;
22     }
23 }
```

فقط منطق fill طبق توضیحی که دادیم فرق می‌کرد و بقیه کاملاً مشابهند.

۲-۹ DRRIP

منطق انتخاب بلوک قربانی:

در روش DRRIP (Dynamic RRIP) منطق خارج کردن بلوک دقیقاً مشابه روش SRRIP است و تفاوتی ندارد.

نحوه بروزرسانی اطلاعات کمکی:

این روش هم چون یک روش تکمیلی روی SRRIP است تقریباً مشابه عمل می‌کند به جز قراردادی که در هنگام اضافه کردن بلوک به set دارد. به این صورت عمل می‌کند که دو تا sub-policy دارد که بین این دو تا به صورت پویا عوض می‌کند. sub-policy اول به این صورت است که برای بلوک جدیدی که اضافه می‌شود مثل SRRIP برای مقدار RRPV آن MAX_RRPV-1 را قرار دهیم. و sub-policy دیگر این است که مثل BIP، با احتمال خیلی زیاد (ممکن است پایاده سازی این مورد به صورت احتمالی نباشد، مثلاً در پایاده سازی ما اینطور نیست) مقدار RRPV آن را MAX_RRPV قرار دهیم (مرز خروجی). و با احتمال خیلی کمتر MAX_RRPV-1 را قرار دهیم. در این روش اینکه کدام یک از این sub-policy ها انتخاب می‌شود با استفاده از روش set-dueling انجام می‌شود. به این صورت که ما ۳ نوع دسته داریم: brrip_leader و srrip_leader و follower. برای دسته های brrip_leader و srrip_leader به صورت ثابت

به ترتیب از sub-policy های SRRIP و BRRIP استفاده می کنیم ولی برای دسته آخر که follower است با توجه به یک شمارنده PSEL انجام می شود.

هنگام hit هم فارغ از اینکه sub-policy کدام باشد مقدار RRPV به ۰ تنظیم می شود.

نکات کد:

۲-۹-۱ قطعه ۱ – ثابت ها و شمارنده ی PSEL

```
1 struct drrip : public champsim::modules::replacement {
2     static constexpr unsigned maxRRPV = 3;
3     static constexpr unsigned BRRIP_MAX = 32;
4     static constexpr unsigned PSEL_WIDTH = 10;
5
6     enum class set_type { follower, brrip_leader, srrip_leader };
7
8     unsigned brrip_counter;
9
10    std::vector<unsigned> rrpv;
11    std::vector<champsim::msl::dscounter<long, PSEL_WIDTH>> PSEL;
12 };
13
14 drrip::drrip(CACHE* cache)
15     : replacement(cache), NUM_SET(cache->NUM_SET), NUM_WAY(cache->NUM_WAY),
16       rrpv(static_cast<std::size_t>(NUM_SET * NUM_WAY)),
17       PSEL(NUM_CPUS, champsim::msl::dscounter<long,
18         PSEL_WIDTH>(champsim::msl::get_sample_rate(NUM_SET)))
19 { }
```

یک شمارنده ی PSEL با طول ۱۰ بیت به ازای هر هسته داریم. ۳ دسته برای set ها طبق توضیحی که دادیم داریم. و بقیه ی المان های SRRIP و المان های مشابه BIP را نیز مشاهده می کنیم.

۲-۹-۲ قطعه ۲ – دو sub-policy درج

```
1 void drrip::update_brrip(long set, long way)
2 {
3     get_rrpv(set, way) = maxRRPV;
4
5     brrip_counter++;
6     if (brrip_counter == BRRIP_MAX) {
7         brrip_counter = 0;
8         get_rrpv(set, way) = maxRRPV - 1;
9     }
10 }
11
12 void drrip::update_srrip(long set, long way) { get_rrpv(set, way) = maxRRPV - 1; }
```

در حالت BRRIP بلوک جدید در ۳۱ حالت از هر ۳۲ حالت با مقدار maxRRPV اضافه می‌شود (اینجا یک تفاوت ریز با BIP دارد و آن این است که در BIP این موضوع رندوم بود ولی اینجا به صورت چرخشی و سیکلی است و رندوم نیست، تاثیر عملکردی آن شاید ناچیز باشد و تقریباً یکسان باشند ولی از لحاظ سخت افزاری خیلی ساده تر از رندوم است)، یعنی در هر خروج قرار می‌گیرد و اگر تا اولین جستجوی قربانی استفاده نشود بیرون می‌رود. فقط یک بار در هر ۳۲ درج، مقدار maxRRPV - 1 داده می‌شود تا cache بتواند به تدریج محتوای جدید را نگه دارد.

تابع update_srrip دقیقاً همان درج SRRIP است و هیچ تفاوتی با آن ندارد.

۳-۹-۲ قطعه ۳ – انتخاب sub-policy در زمان درج

```
1 void drrip::replacement_cache_fill(uint32_t triggering_cpu, long set, long
  way, ...)
2 {
3     // do not update replacement state for writebacks
4     if (access_type{type} == access_type::WRITE) {
5         get_rrpv(set, way) = maxRRPV - 1;
6         return;
7     }
8     if (PSEL[triggering_cpu].decide(set)) {
9         update_brrip(set, way);
10    } else {
11        update_srrip(set, way);
12    }
13    // cache miss, update bad
14    PSEL[triggering_cpu].update_bad(set);
15 }
```

تابع decide(set) برای set های leader همیشه سیاست فیکس شده ی خودشان را برمی گرداند و برای set های follower بر اساس مقدار فعلی PSEL تصمیم می‌گیرد.

تابع update_bad(set) در هر fill صدا زده می‌شود و چون fill فقط بعد از miss اتفاق می‌افتد، عملاً هر miss در یک set رهبر، PSEL را به ضرر سیاست همان رهبر تغییر می‌دهد. در نتیجه بعد از مدتی PSEL به سمت سیاستی می‌رود که در set های نمونه miss کمتری داده است. این همان بخش پویای این روش است. اینکه خود تابع update_bad چطور پیاده سازی شده است در فایل های داخلی ChampSim است و خیلی جزیی می‌شود.

۴-۹-۲ قطعه ۴ – hit و انتخاب قربانی

```
1 void drrip::update_replacement_state(..., uint8_t hit)
2 {
3     if (hit) {
4         if (access_type{type} == access_type::WRITE) {
5             get_rrpv(set, way) = maxRRPV - 1;
6             return;
7         }
8         get_rrpv(set, way) = 0; // DRRIP always promotes a cache line to the MRU
          position
9     }
10 }
```

```

11
12 long drrip::find_victim(uint32_t triggering_cpu, uint64_t instr_id, long set, ...)
13 {
14     auto begin = std::next(std::begin(rrpv), set * NUM_WAY);
15     auto end = std::next(begin, NUM_WAY);
16
17     auto victim = std::max_element(begin, end);
18     if (auto rrpv_update = maxRRPV - *victim; rrpv_update != 0)
19         for (auto it = begin; it != end; ++it)
20             *it += rrpv_update;
21
22     return std::distance(begin, victim);
23 }
    
```

تابع `find_victim` از نظر منطقی دقیقاً همان SRRIP است و فقط به جای `std::transform` از یک حلقه استفاده کرده است.

در زمان برخورد، بدون توجه به اینکه کدام sub-policy فعال است، مقدار RRPV به ۰ می‌رسد.

SHiP ۲-۱۰

منطق انتخاب بلوک قربانی:

در روش SHiP (Signature-based Hit Predictor)، کاملاً مشابه روش SRRIP مقادیر RRPV به ازای هر بلوک داریم که دقیقاً به همان روش SRRIP بلوک هدف را خارج می‌کنیم.

نحوه بروزرسانی اطلاعات کمکی:

در این policy ما روی مقدار اولیه RRPV که به ازای هر بلوک نسبت می‌دهیم خلاقیت به خرج می‌دهیم و سعی می‌کنیم مقدار هوشمندانه‌ای برای آن تعیین کنیم. به این صورت که در کنار RRPV یک تگ به اسم `signature` هم به هر بلوک هنگام اضافه شدن به `set` به آن می‌دهیم که بر اساس ناحیه‌ی حافظه‌ای است که در آن قرار داشته است یا آدرس حافظه مربوط به آن است (PC) (کلاً قرارداد مخصوص به خود دارد، مثلاً در کد ما همان PC است). یک جدول به اسم SHCT (Signature History Counter Table) نگه می‌داریم که به ازای هر `signature` یک شمارنده دارد که در هر بار دسترسی به بلوک با آن `signature` خاص شمارنده مربوط به `signature` در جدول کم می‌شود. عملاً مقدار به ازای هر `signature` در این جدول بیان می‌کند که این پترن خاص از حافظه چقدر امکان مجدد بازخواست و دسترسی دارند. هنگام اضافه کردن بلوک به `set` از این جدول `lookup` انجام می‌دهیم و مقدار مربوط به `signature` این بلوک را می‌خوانیم، بر اساس این مقدار RRPV را مقداردهی می‌کنیم. در صورتی که این مقدار کم باشد و یعنی احتمال بازخواست این `signature` زیاد باشد مقدار RRPV آن را یک چیز کم مثل ۰ یا ۱ می‌دهیم (البته اینکه در حالت خوب یک مقدار RRPV به آن داده شود در کد نیست و در اولین پیاده‌سازی‌های این ایده هم ظاهراً نبوده و در مقالات و ورژن‌های بعدی که بهبود یافته است ظاهر می‌شود). و اگر مقدار جدول زیاد باشد و یعنی این `signature` احتمالاً زیاد بازخواست نمی‌شود مقدار RRPV بالایی مثل `MAX_RRPV` به آن نسبت می‌دهیم.

در هنگام `hit` مقدار RRPV بلوک به ۰ ریست می‌شود و ورودی داخل جدول SHCT که مربوط به `signature` این بلوک است یکی کم می‌شود.

در هنگامی که بلوک بدون اینکه یکبار هم `hit` شده باشد در جدول مقدار مربوط به `signature` آن یکی زیاد می‌شود.

نکته‌ی بسیار مهم این است که این جدول اگر تمام `set`‌های موجود ذخیره شود سخت افزار خیلی بزرگی می‌خواهد. پس به جای اینکار ما این را برای تعداد محدودی `set` انجام می‌دهیم و به بقیه `set`‌ها تعمیم می‌دهیم. به این `sampler` می‌گوییم که چیزی شبیه شبکه عصبی که با یک سری `set`‌ها `train` می‌شود و روی بقیه تست می‌شود. به این شکل سخت افزار کمتری مصرف می‌شود. در جاهای مختلف کد این موضوع تکرار شده ولی چون از توضیحی که دادیم خارج نیست، فرض می‌کنیم کلاً `sampler` را ندید می‌گیریم و کلی توضیح می‌دهیم.

پس داخل کد و توضیحات بیشتر جاها منظورمان خود sampler است که در یادگیری استفاده می شود ولی برای ساده سازی به خود جدول اشاره کردیم.

نکات کد:

۱-۱۰-۲ قطعه ۱ – ساختار sampler و جدول SHCT

```
1 struct ship : public champsim::modules::replacement {
2     static constexpr int maxRRPV = 3;
3     static constexpr std::size_t SHCT_SIZE = 16384;
4     static constexpr unsigned SHCT_PRIME = 16381;
5     static constexpr unsigned SHCT_MAX = 7;
6
7     class SAMPLER_class
8     {
9     public:
10         bool valid = false;
11         bool used = false;
12         champsim::address address{};
13         champsim::address ip{};
14         uint64_t last_used = 0;
15     };
16
17     std::vector<SAMPLER_class> sampler;
18     std::vector<int> rrpv_values;
19     champsim::msl::categorizer<long> set_categorizer;
20     champsim::data::bits sampler_tag_bits;
21
22     std::vector<std::array<champsim::msl::fwcounter<champsim::msl::lg2(SHCT_MAX +
23         1)>, SHCT_SIZE>> SHCT;
```

ساختار داده SHCT یک جدول با ۱۶۳۸۴ خانه است که هر خانه ی آن یک شمارنده ۳ بیتی است (چون SHCT_MAX = 7 است).

هر خانه ی sampler یک بیت used دارد که می گوید آیا آن بلوک بعد از درج حداقل یک بار استفاده شد یا نه.

۲-۱۰-۲ قطعه ۲ – آموزش جدول از روی sampler

```
1 void ship::update_replacement_state(..., uint8_t hit)
2 {
3     // update sampler
4     if (set_categorizer.get_sample_category(set) == 0) {
5         auto s_idx = set / champsim::msl::get_sample_rate(NUM_SET);
6         auto s_set_begin = std::next(std::begin(sampler), s_idx * NUM_WAY + ...);
7         auto s_set_end = std::next(s_set_begin, NUM_WAY);
8     }
```



```

9      // check hit
10     auto match = std::find_if(s_set_begin, s_set_end, [addr = full_addr, shamt =
sampler_tag_bits](auto x) {
11         return x.valid && x.address.slice_upper(shamt) == addr.slice_upper(shamt);
12     });
13     if (match != s_set_end) {
14         auto SHCT_idx = match->ip.slice_lower<32_b>().to<std::size_t>() % SHCT_PRIME;
15         SHCT[triggering_cpu][SHCT_idx] -= 1;
16         match->used = true;
17     } else {
18         match = std::min_element(s_set_begin, s_set_end, [](auto x, auto y) { return
x.last_used < y.last_used; });
19
20         if (!match->used) {
21             auto SHCT_idx = match->ip.slice_lower<32_b>().to<std::size_t>() %
SHCT_PRIME;
22             SHCT[triggering_cpu][SHCT_idx] += 1;
23         }
24
25         match->valid = true;
26         match->address = full_addr;
27         match->ip = ip;
28         match->used = false;
29     }
30
31     // update LRU state
32     match->last_used = access_count++;
33 }
34
35 if (hit)
36     get_rrpv(set, way) = 0;
37 }

```

در صورتی که hit داشته باشیم از شمارنده مربوط به آن signature یکی کم می شود و اگر بدون اینکه از بلوک اضافه شده استفاده می مجدد شود و evict شود شمارنده را یکی زیاد می کنیم.

بخش signature همان ۳۲ بیت پایین PC است که با % SHCT_PRIME روی جدول می می شود.

بخش sampler برای خودش یک سیاست LRU مستقل دارد که با last_used و access_count پیاده شده و هیچ ربطی به RRPV های کش اصلی ندارد. یعنی sampler یک کش مجازی کوچک است که فقط برای آموزش نگه داشته می شود.

در صورت hit در cache مقدار RRPV را به ۰ ریست می کند.

۲-۱۰-۳ قطعه ۳ – استفاده از پیش بینی در زمان درج

```

1 void ship::replacement_cache_fill(uint32_t triggering_cpu, long set, long
  way, ...)
2 {
3     // handle writeback access
4     if (access_type{type} == access_type::WRITE) {
5         get_rrpv(set, way) = maxRRPV - 1;
6         return;
7     }
8
9     // SHIP prediction
10    auto SHCT_idx = ip.slice_lower<32_b>().to<std::size_t>() % SHCT_PRIME;
11
12    get_rrpv(set, way) = maxRRPV - 1;
13    if (SHCT[triggering_cpu][SHCT_idx].is_max())
14        get_rrpv(set, way) = maxRRPV;
15 }

```

همان بخش پویا که بر اساس مقدار جدول تصمیم می‌گیرد که مقدار اولیه RRPV چه باشد. حالت پیشفرض مثل SRRIP مقدار $\text{maxRRPV} - 1$ است و حالت خیلی بد مقدار maxRRPV را به خود می‌گیرد. شرط `is_max` همان بررسی حالت خیلی بد است که در صورتی که شمارنده پر شده باشد (یعنی مقدار ۰ را به خود گرفته باشد) فعال می‌شود.

۲-۱۰-۴ قطعه ۴ – انتخاب قربانی

```

1 long ship::find_victim(uint32_t triggering_cpu, uint64_t instr_id, long
  set, ...)
2 {
3     auto begin = std::next(std::begin(rrpv_values), set * NUM_WAY);
4     auto end = std::next(begin, NUM_WAY);
5
6     auto victim = std::max_element(begin, end);
7     if (auto rrpv_update = maxRRPV - *victim; rrpv_update != 0)
8         for (auto it = begin; it != end; ++it)
9             *it += rrpv_update;
10
11    return std::distance(begin, victim);
12 }

```

این تابع عیناً همان `find_victim` سیاست های SRRIP و DRRIP است.

توضیح Trace ها

در رابطه با trace ها انتخاب شده: trace اول 403.gcc بود و برای آن انتخابش کردیم که همجواری زمانی بالایی داشت. علت هم آن است که مربوط به کامپایلر gcc است کامپایلر بار ها به ساختار های داده یکسان (مثل جدول نماد ها و ...) دسترسی می‌خواهد بنابراین بلوک های کشی که به تازگی به از آنها استفاده شده، احتمالا دوباره از آنها استفاده خواهد شد و همانطور هم که در نمودار ها مشخص است و مورد انتظار ماست، سیاست LRU عملکرد خیلی خوبی روی آن داشته است.

ردپای دوم 429.mcf بود و علت انتخاب این بود که همجواری مکانی بالایی داشت. علت این اتفاق هم این است که این ردپا مربوط به حل مسایل بهینه سازی شبکه است و حجم خیلی زیادی داده را در حافظه پردازش میکند. این دسترسی ها معمولا به آدرس های مجاور هستند و برای همین همجواری مکانی در آن مهم است. یک نکته دیگر هم این است که به خاطر دسترسی های زیاد به حافظه، تاثیر اندازه کش در آن به خوبی محسوس است که در نمودار ها هم قابل مشاهده است.

ردپای سوم 470.lbm بود و علت انتخاب این بود که رفتار جریانی را درون خود دارد. این برنامه مربوط به شبیه سازی دینامیک سیالات است و داده ها را عمدتا به صورت ترتیبی و یک بار مصرف پردازش می کند. بنابراین پس از دسترسی به یک بلوک حافظه، احتمال استفاده مجدد از آن بسیار کم است. بنابراین این ردپا نمونه ی خوبی از رفتار جریانی محسوب می شود و همانطور که در نمودار ها واضح است hit rate برای برخی از سیاست ها بسیار پایین است که مطابق انتظار ماست.

ردپای آخر هم 471.omnetpp بود و برای این انتخاب شد که بین سیاست های جایگزینی تفاوت قابل قبولی قابل میشد. چرا که این ردپا مربوط به شبیه سازی شبکه های کامپیوتری است. الگوی دسترسی آن به نسبت پیچیده است و مجموعه داده های فعال آن به گونه ای است که انتخاب بلوک برای جایگزینی می تواند تاثیر قابل توجهی بر نرخ برخورد و عملکرد سیستم داشته باشد. این مورد هم دقیقا روی نمودار ها قابل مشاهده است و بین سیاست ها تفاوت وجود دارد.

نکته دیگر درباره شبیه سازی ها این است که تمام ۹۴ شبیه سازی، با ۱۲۵۰۰۰۰۰۰ دستور گرم سازی و ۵۰۰۰۰۰۰۰۰ دستور شبیه سازی انجام شد. این اتفاق خوبی بود که تمام شبیه سازی ها با تعداد دستور یکسان اجرا شدند زیرا باعث عدالت شد و تعداد دستورات هم مقدار زیادی بود تا به خوبی سیاست های مختلف روی ردپا های مختلف مورد آزمایش قرار بگیرند و تفاوت میان ردپا ها قابل حس باشد ولی طبیعتا به خاطر اینکه ردپا های مختلف داریم، تفاوت زمانی بین شبیه سازی ها دیده میشود. مثلا شبیه سازی ها با این تعداد دستور روی ردپای 429.mcf حدود ۲۰+ دقیقه زمان میبرد در حالی که بقیه شبیه سازی ها معمولا زیر ۱۰ دقیقه بودند و علت آن هم طبیعتا نوع متفاوت ردپاهاست و همانطور که گفته شده بود ردپای 429.mcf خیلی به حافظه دسترسی دارد (تعداد دسترسی ها همانطور که در llc_results مشاهده میشود برای این ردپا بیشتر از بقیه است) بنابراین نسبت به بقیه ردپا ها خیلی بیشتر در llc خطا داریم و از آنجا که خطا در llc پهنالتی خیلی زیادی دارد، باعث طولانی تر شدن شبیه سازی در این ردپا در مقایسه با بقیه شده است. میانگین دسترسی به llc در بقیه نزدیک به هم هستند و فقط 471.omnetpp مقدار کمی از بقیه کمتر به llc دسترسی دارد و همین مورد هم سبب این شده که این ردپا هم کمی نسبت به بقیه زمان کمتری بگیرد.

بخش ۴

نحوه ی اجرا و خروجی گرفتن پروژه

توضیح script ها و ...

یادداشت

برای گام های اولیه sets=2048 دیفالت باشد.
تمام فعل ها و اشاره ها باید به صورت «ما» باشد نه «من».

مراجع
